

Mathematics: analysis and approaches HL | Exploration

How does the choice of optimization algorithm, such as gradient descent or simulated annealing, affect the accuracy and efficiency of machine learning-based regression analysis?

Candidate number: jqx050

Session: May 2024

Page count: x

Contents

1 Introduction	1
2 Methodology	1
2.1 Regression and optimization	1
2.2 Loss functions	2
2.3 Optimization algorithms	2
2.4 Modeling the problem	3
3 Gradient descent (GD)	4
3.1 The gradient descent algorithm	4
3.2 Gradient descent on our problem	9
4 Simulated annealing (SA)	10
4.1 The simulated annealing algorithm	10
4.2 Simulated annealing on our problem	13
5 Analysis	14
6 Conclusion	16
Bibliography	17
Appendix I: Dataset excerpt	18
Appendix II: Data processing code and raw outputs	18

1 | Introduction

Machine learning (ML) is a branch of artificial intelligence that uses statistical techniques to give computer systems the ability to “learn” from and make predictions on data, without being explicitly programmed. Today, machine learning models are used in our everyday lives, from personalized recommendations on Netflix to facial recognition on our phones.

The very first machine learning task I learned to solve was **regression analysis**, which is the statistical process of finding a correlation between a dependent variable and one or more independent variables (Freedman, 2009). This is often used to predict future values of the dependent variable based on values of the independent variable(s), for instance predicting the price of a house based on its size and location. The process of training a regression model involves **optimization**. This optimization can be done using a variety of algorithms, but the end goal is always the same: to find the best parameters of the model by minimizing the difference between the model’s predictions and the actual values of the dependent variable.

In that study of machine learning, I have come to value understanding the underlying algorithms, because knowing the limitations and strengths of different algorithms can help data scientists choose the right one for a given problem. For this reason, this exploration will focus on how the choice of **optimization algorithm** affects the **accuracy** and **efficiency** of machine learning-based regression analysis, which in the real world translates to how accurate a model’s predictions are on new data, and how quickly it can be “trained” to do so.

2 | Methodology

2.1 | Regression and optimization

In applied statistics (which blends mathematics and computer science), **regression** is a **supervised learning**¹ algorithm that models the relationship between a dependent variable y and an independent variable(s) x . The model can be represented visually as a line or curve that best fits the data.

To compute this regression, we optimize. **Optimization** is the process of finding the best solution to a problem, which for our problem is finding the equation of the line that best fits the data, according to some measure. In machine learning, this measure is called a loss function; and so the goal of optimization becomes minimizing the **loss function**.

¹**Supervised learning** is a category of machine learning tasks where a model learns a function based on example input-output pairs (from the **training data**).

2.2 | Loss functions

There are many types of loss functions, which depend on the problem and type of data. This includes “square of Euclidean distance, cross-entropy, contrast loss, hinge loss, information gain,” etc (Sun et al., 2019).

A common loss function is the sum of squared residuals, defined as

$$L = \sum_{i=1}^n (y_i - f(x_i))^2 \quad (1)$$

where y_i is the i^{th} actual value of the variable we are trying to predict (“ground truth”), $f(x_i)$ is the predicted value of y_i given the i^{th} value of the independent variable x , and n is the number of data points. In this case, y is the fuel efficiency of the car, x is the horsepower rating of the car, and $f(x)$ is the predicted fuel efficiency of the car given its horsepower rating.

This loss function was chosen because it is a common loss function for regression problems, and is differentiable, which is a requirement for gradient-based methods.

2.3 | Optimization algorithms

Optimization algorithms are used to minimize the loss function and in turn, solve the optimization problem. While they can work on varying principles, they can broadly be categorized into **deterministic** and **stochastic** algorithms. Deterministic algorithms are those that follow a fixed set of rules to find the minimum of the loss function, while stochastic algorithms use randomness to find the minimum (Sun et al., 2019).

To look at the effect of each optimization algorithm on the performance of a regression model, we will apply both a deterministic and a stochastic algorithm to a real-world dataset of information on cars. We will then compare the results of the two algorithms to an analytical solution (see Section 5) to determine which algorithm is more accurate and efficient.

An **accurate** regression model for the problem would help us accurately predict the fuel efficiency of a car given its horsepower, both interpolating and extrapolating. This can be measured statistically, with the **coefficient of determination** R^2 , as well as in comparison to the analytical solution.

An **efficient** regression model for the problem would be able to find the best-fit line with as few iterations as possible. This can be measured statistically by the number of iterations taken to converge on a solution, as well as the rate of convergence.

2.4 | Modeling the problem

“Car information dataset” is a dataset published on Kaggle, a data aggregator, by Tawfik Elmetwally in 2023 (Elmetwally, 2023). The dataset contains information on 399 cars, including their make, model, engine displacement and cylinder count, horsepower, fuel efficiency, and more. We will use this dataset to model the relationship between horsepower and fuel efficiency.

Extracting the relevant columns (mpg and horsepower) from the dataset and plotting them on a set of axes, we get the following scatterplot:

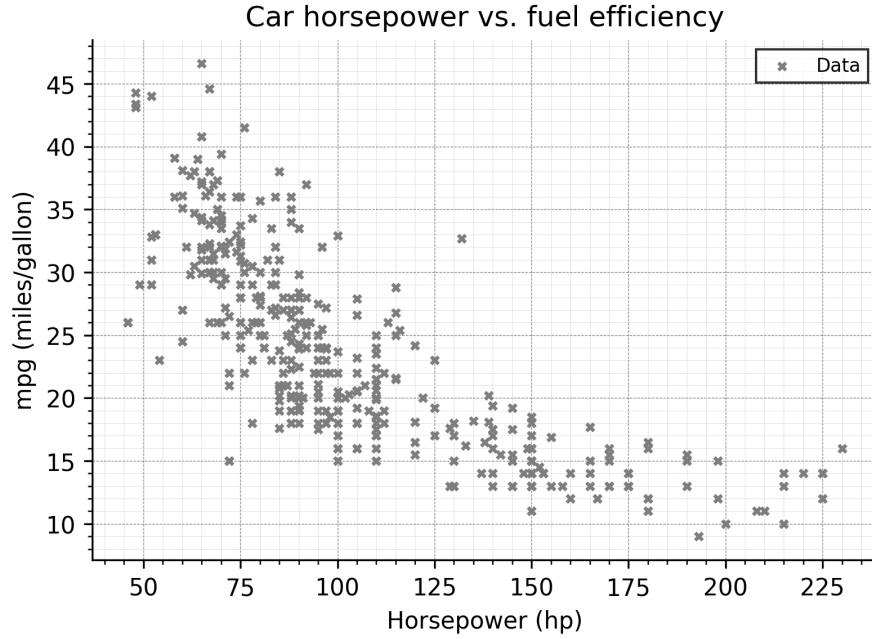


Figure 1: Scatterplot of car data. Derived from dataset published by Elmetwally (2023).

Let us define our problem to be modeling this relationship between the horsepower rating (in hp) of a car and its fuel efficiency (in mpg).

From the scatterplot, we observe a negative exponential relationship between the two variables, where as horsepower increases, fuel efficiency decreases at a slowing rate. Hence, we will attempt to optimize an exponential regression model to fit the data.

This will be in the form of the equation

$$f(x) = a \cdot e^{b(x-h)} + k.$$

Parameter	Domain	Range
$f(x)$	$x \in \mathbb{R}$	$f(x) > k$

Parameter	Meaning	Domain
a	Scaling factor	$a \in \mathbb{R}$
b	Growth factor	$b \in \mathbb{R}$
h	Horizontal shift	$h \in \mathbb{R}$
k	Vertical shift	$k \in \mathbb{R}$

3 | Gradient descent (GD)

Gradient descent is an optimization algorithm that minimizes a function by taking steps proportional to the negative of the gradient of the function at the current point.

For convex loss functions with a clear global minimum, gradient descent should be powerful and efficient, because it will deterministically find the minimum of the loss function. However, in non-convex loss functions or ones that have multiple local minima, it can be slow to converge or get “stuck” in local minima and never converge on the global minimum (Sun et al., 2019). We will soon infer whether this is the case for our problem.

3.1 | The gradient descent algorithm

Each squared residual comes from the difference between the actual value of y and the predicted value of y , $f(x_i)$. Therefore, the loss function can be rewritten as

$$L = \sum_{i=1}^n (y_i - a \cdot e^{b(x-h)} - k)^2 \quad (2)$$

When we optimize this loss function, we can calculate the summation, substituting y_i and x_i with each datapoint’s value, which leaves the loss function in terms of the parameters a , b , h , k as the targets for optimization.

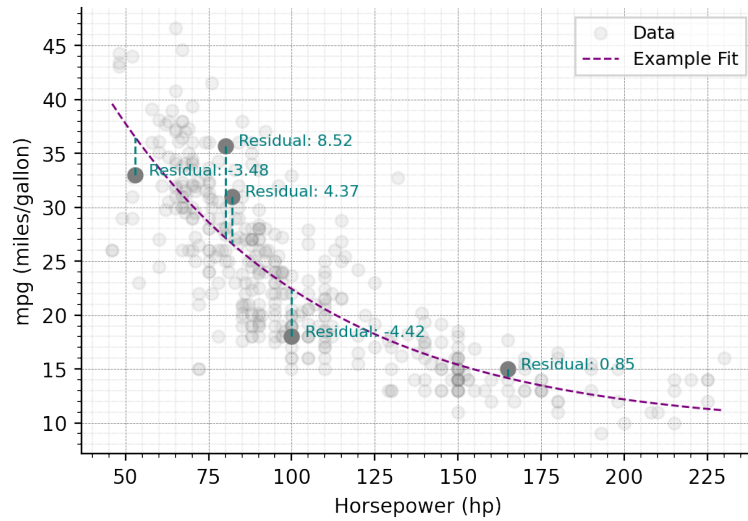


Figure 2: As an example, residuals between datapoints and an example regression line

Initialization

We first define an initial solution (a mathematical “**guess**” based on intuition and visual data), where we choose arbitrary but sensible values for the parameters. Let’s call these a_0 , b_0 , h_0 , and k_0 . In choosing guesses, guesses closer to the actual values will allow the optimization algorithm to converge on a solution in fewer iterations, so the same guess will be used for GD and SA.

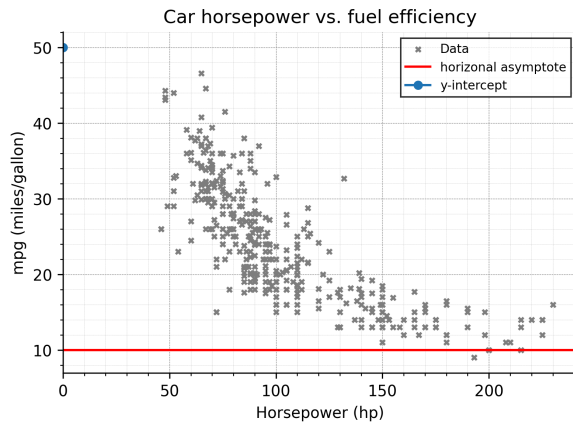


Figure 3: Visual estimation of parameters

→ Let’s choose:

- $a_0 = 1$
- $b_0 = -0.05$, as we recognize an exponential decay shape.
- $h_0 = 80$, as that brings the y-intercept around 50 mpg.
- $k_0 = 10$, as the horizontal asymptote seems to exist around 10 mpg.

These parameter values will be continually adjusted in each iteration of the algorithms, and the final values after the loss function converges can be substituted into the original identity function $f(x)$ to get the equation of the regression.

Computing the gradient

Taking the partial derivative of the loss function with respect to each parameter gives us the instantaneous rate of change of each parameter, which is the slope of the loss function at that point. Partial derivatives are used because the loss function is in terms of 4 parameters, and we want to find the gradient of each while the others are held constant.

Together, the partial derivatives make up the **gradient** of the loss function, which “denotes the direction of greatest change of a scalar function” (Bachman, 2007) — essentially a vector that points in the direction of the steepest ascent of the loss function. The idea with GD is that, by taking steps in the opposite direction of the gradient, we will eventually reach the minimum of the loss function, and in the process obtain the parameters a , b , h , and k that provide that minimum. This is our optimal solution.

Following this, we compute the derivatives.

→ **Derivative with respect to a**

From Equation 2, we have

$$L(a, b, h, k) = \sum (y_i - a \cdot e^{b(x_i-h)} - k)^2$$

Differentiating L with respect to a , we can apply the chain rule.

$$\begin{aligned} \frac{\partial L}{\partial a} &= 2 \times (y_i - a \cdot e^{b(x_i-h)} - k) \times -e^{b(x_i-h)} \\ \frac{\partial L}{\partial a} &= -2 \times \sum e^{b(x_i-h)} (y_i - a \cdot e^{b(x_i-h)} - k) \end{aligned} \quad (3)$$

Following the same mathematical approach, the derivatives with respect to b , h , and k can be computed:

→ **Derivative with respect to b**

$$\begin{aligned} \frac{\partial L}{\partial b} &= 2 \times (y_i - a \cdot e^{b(x_i-h)} - k) \times a \cdot e^{b(x_i-h)} \cdot (x_i - h) \\ \frac{\partial L}{\partial b} &= -2 \times \sum a \cdot e^{b(x_i-h)} (x_i - h) (y_i - a \cdot e^{b(x_i-h)} - k) \end{aligned} \quad (4)$$

→ **Derivative with respect to h**

$$\begin{aligned} \frac{\partial L}{\partial h} &= 2 \times (y_i - a \cdot e^{b(x_i-h)} - k) \times -ab \cdot e^{b(x_i-h)} \\ \frac{\partial L}{\partial h} &= 2 \times \sum ab \cdot e^{b(x_i-h)} (y_i - a \cdot e^{b(x_i-h)} - k) \end{aligned} \quad (5)$$

→ **Derivative with respect to k**

$$\begin{aligned} \frac{\partial L}{\partial k} &= 2 \times (y_i - a \cdot e^{b(x_i-h)} - k) \times -1 \\ \frac{\partial L}{\partial k} &= -2 \times \sum (y_i - a \cdot e^{b(x_i-h)} - k) \end{aligned} \quad (6)$$

Altogether, the gradient is

$$\begin{bmatrix} \frac{\partial L}{\partial a} \\ \frac{\partial L}{\partial b} \\ \frac{\partial L}{\partial h} \\ \frac{\partial L}{\partial k} \end{bmatrix} = \begin{bmatrix} -2 \times \sum e^{b(x_i-h)} (y_i - a \cdot e^{b(x_i-h)} - k) \\ -2 \times \sum a \cdot e^{b(x_i-h)} (x_i - h) (y_i - a \cdot e^{b(x_i-h)} - k) \\ 2 \times \sum ab \cdot e^{b(x_i-h)} (y_i - a \cdot e^{b(x_i-h)} - k) \\ -2 \times \sum (y_i - a \cdot e^{b(x_i-h)} - k) \end{bmatrix} \quad (7)$$

This current parameters are then substituted into the gradient.

→ For instance, substituting the initial parameters a_0 , b_0 , h_0 , and k_0 into the gradient and computing the summation over every data point, we get the following value:

$$\begin{bmatrix} 15921.6831 \\ 87049814.8282 \\ 796.0842 \\ 17774.9488 \end{bmatrix} \quad (8)$$

Parameter update

The gradient is not used directly, however. Rather, the gradient is multiplied by a scalar, the **learning rate** (γ), which controls the rate of parameter change. A balance must be found with the learning rate, as a learning rate too small “will result in a slower convergence rate”, while a learning rate too large can cause overshooting, hindering convergence (Sun et al., 2019). The product of the gradient and learning rate is the **step size**, which is the actual amount each parameter is changed per iteration.

$$\text{Step size} = \gamma \times \begin{bmatrix} \frac{\partial L}{\partial a} \\ \frac{\partial L}{\partial b} \\ \frac{\partial L}{\partial h} \\ \frac{\partial L}{\partial k} \end{bmatrix} \quad (9)$$

The step sizes are then subtracted from the original parameters (the parameters from the previous iteration or the initial values) to get new values. If n is the current iteration, this can be defined as

$$\begin{bmatrix} a_n \\ b_n \\ h_n \\ k_n \end{bmatrix} = \begin{bmatrix} a_{n-1} \\ b_{n-1} \\ h_{n-1} \\ k_{n-1} \end{bmatrix} - \gamma \times \begin{bmatrix} \frac{\partial L}{\partial a} \\ \frac{\partial L}{\partial b} \\ \frac{\partial L}{\partial h} \\ \frac{\partial L}{\partial k} \end{bmatrix} \quad (10)$$

→ Continuing the example iteration, let's set the learning rate γ to 10^{-9} initially (from Sun et al. (2019)). Following the steps outlined above, we calculate the step size and then the new values for each parameter.

$$\text{Step size} = 10^{-9} \times \begin{bmatrix} 15921.6831 \\ 87049814.8282 \\ 796.0842 \\ 17774.9488 \end{bmatrix} \quad (11)$$

and the new parameter values (since we don't have a previous iteration, we use the initial values for iteration $n - 1$):

$$\begin{bmatrix} a_n \\ b_n \\ h_n \\ k_n \end{bmatrix} = \begin{bmatrix} 1 \\ -0.05 \\ 80 \\ 10 \end{bmatrix} - 10^{-9} \times \begin{bmatrix} 15921.6831 \\ 87049814.8282 \\ 796.0842 \\ 17774.9488 \end{bmatrix} = \begin{bmatrix} 0.999984078317 \\ -0.13704981428 \\ 79.9999992039 \\ 9.99998222505 \end{bmatrix} \quad (12)$$

After one iteration of GD, we now have a refinement of our regression equation:

$$f(x) = 0.999984078317 \times e^{-0.13704981428(x-79.9999992039)} + 9.99998222505 \quad (13)$$

Over this single iteration, the changes are minuscule, but over many iterations, the loss function eventually converges to a minimum.

Convergence check and iteration

This process of calculating the gradient, step size, and finally the new parameter values, is repeated until the loss function converges to a minimum, where the loss function approaches a value that is either the global minimum or a local minimum, at which further adjustments to the model parameters result in negligible or no improvement in the loss function.

Convergence can be determined by setting a threshold for the step size (when it is sufficiently small) or by limiting the number of iterations.

3.2 | Gradient descent on our problem

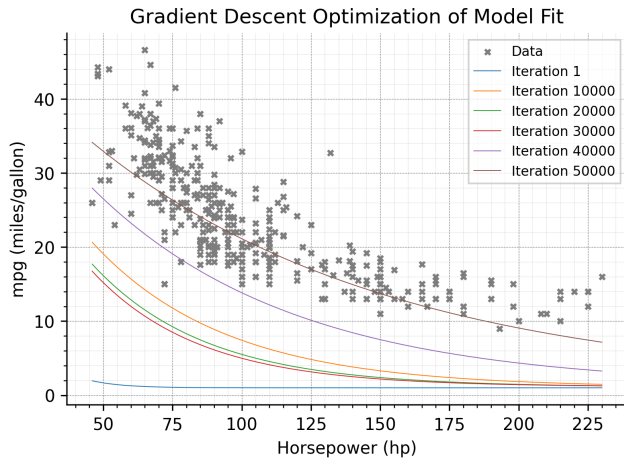


Figure 4: Gradient descent on our problem

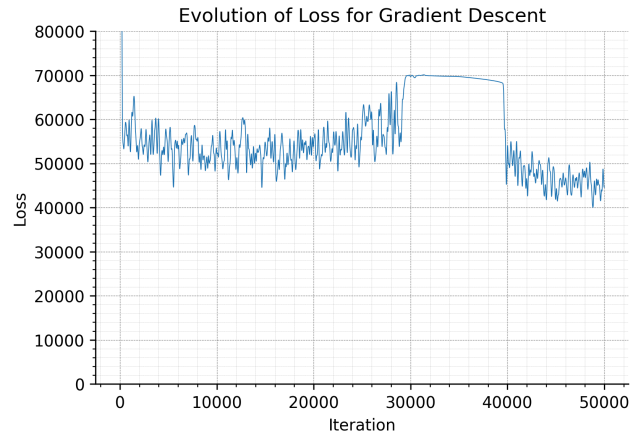


Figure 5: Loss function over iterations

Programmatically applying the gradient descent process on the data, the algorithm ran through 50,000 iterations, which is visualized above.

While the difference between each iteration is minimal, these changes lead to the parameters slowly converging over many iterations on a satisfactory solution (brown line, Iteration 50000) that appears to fit the scatterplot. This is reflected in the loss graph, which decreases over time although there is significant noise.

The solution from Iteration 50,000 yielded an R^2 value (coefficient of determination) of 0.603810, which statistically indicates approximately 60.38% of the variance in the dependent variable is explained by the regression model; the model captured a significant portion (but not all) of variability in the data.

4 | Simulated annealing (SA)

In contrast to gradient descent, which is a deterministic gradient-based method, simulated annealing is a probabilistic technique inspired by metallurgy that makes random changes to the solution and uses the concept of ‘temperature’ to control the search process. This should represent a radically different optimization approach: the temperature makes the algorithm more likely to accept “worse” (between iterations) solutions at the beginning of the search process, and more conservative later on when a better approximation to the global minimum of the loss function is found.

4.1 | The simulated annealing algorithm

Initialization

Like in GD, we start with an initial solution, the guess. Let’s use the same guess as before so that the algorithms can be directly compared.

We also set an initially high **temperature** T_0 . The idea is that, at high temperatures, the algorithm is more likely to accept changes that increase the function value (or ‘energy’), allowing it to explore a wide range of solutions. As the temperature decreases, the algorithm becomes more selective, focusing on refining and improving the current solution. This process helps SA to potentially escape local minima and converge towards a global minimum. This balance of exploration (searching new areas of the solution space) and exploitation (refining solutions in promising areas) is useful, especially when the function’s landscape is complex with many local minima (Kirkpatrick et al., 1983).

→ Let’s set the initial temperature T_0 to 500, from Kirkpatrick et al. (1983).

Iteration

In each iteration, we make a random perturbation to the current solution in the form of adding a random value to each parameter. This removes any inherent bias in the direction of the search, allowing for an unbiased exploration of the solution space.

In my implementation, I will generate random numbers from a normal (Gaussian) distribution with $\mu = 0$ and $\sigma = 1$, or $N \sim (0, 1)$, because it is continuous and adjustments in both the positive and negative directions have equal probability.

Hence, if n is the current iteration, the perturbed values are

$$\begin{bmatrix} a_n \\ b_n \\ h_n \\ k_n \end{bmatrix} = \begin{bmatrix} a_{n-1} + \Delta a \\ b_{n-1} + \Delta b \\ h_{n-1} + \Delta h \\ k_{n-1} + \Delta k \end{bmatrix} \quad (14)$$

where $\Delta a \sim N(0, 1)$ and so forth.

→ Generating random perturbations in the program implementation, we get the following perturbed values.

$$\begin{bmatrix} a_n \\ b_n \\ h_n \\ k_n \end{bmatrix} = \begin{bmatrix} 1 + \Delta a \\ -0.05 + \Delta b \\ 80 + \Delta h \\ 10 + \Delta k \end{bmatrix} = \begin{bmatrix} 1.54626094 \\ -0.08207048 \\ 80.84881705 \\ 10.03802701 \end{bmatrix} \quad (15)$$

Evolution and acceptance

After perturbation, the new candidate solution is evaluated by the difference in the loss function, ΔL .

If the candidate solution is better than the current solution, that is, $\Delta L > 0$, then the candidate solution is accepted as the new current solution.

If the candidate solution is worse than the current solution, that is, $\Delta L < 0$, then the candidate solution is accepted as the new current solution with a probability of

$$P(\text{accept}) = e^{-\frac{\Delta L}{T}} \quad (16)$$

where T is the current temperature.

→ $\Delta L = 57943.6433449937$

In this iteration, the candidate solution is better than the current solution, so we accept the candidate solution as the new current solution.

Cooling

At the end of each iteration, the temperature is decreased by a cooling factor α . This is done to reduce the probability of accepting worse solutions as the algorithm progresses.

For instance, a simple exponential decay might be

$$T_{\text{new}} = \alpha \times T_{\text{old}} \quad (17)$$

where α is a constant < 1 , controlling the rate of temperature decrease.

→ Let's choose $\alpha = 0.98$, to give the algorithm more time to explore other minima.

The new temperature is then

$$\begin{aligned} T_{\text{new}} &= 0.98 \times T_{\text{old}} \\ &= 0.98 \times 500 \\ &= 490 \end{aligned} \quad (18)$$

This completes one iteration, which leaves us with a refinement of our regression equation:

$$f(x) = 1.54626094 \times e^{-0.08207048(x-80.84881705)} + 10.03802701 \quad (19)$$

Termination

The algorithm repeats the iteration step until a stopping criterion is met. This could be a sufficiently low temperature, a maximum number of iterations, or a minimal change in the loss function over a certain number of iterations.

4.2 | Simulated annealing on our problem

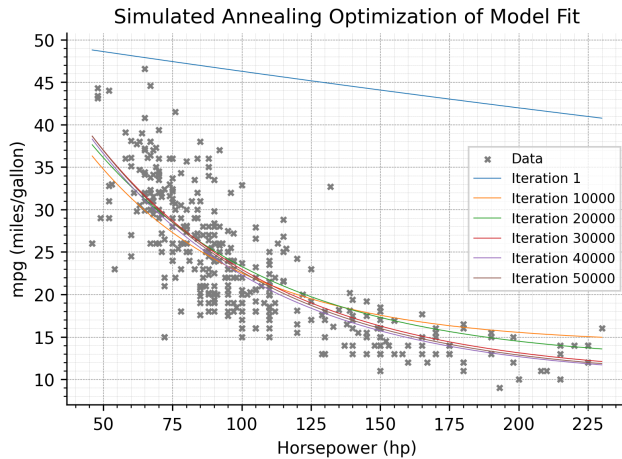


Figure 6: Simulated annealing on our problem

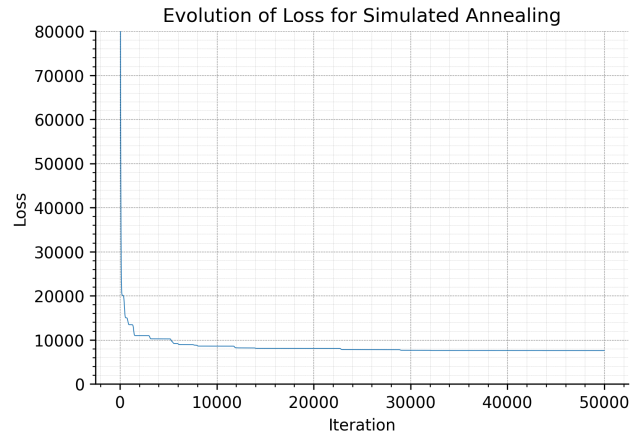


Figure 7: Loss function over iterations

Similarly to the GD algorithm, simulated annealing was implemented programmatically and was run for 50,000 iterations to make a direct comparison.

The loss graph showed a steep drop, indicating that simulated annealing was quickly able to find a satisfactory solution within 10,000 iterations before converging on a minimum (appears as a horizontal asymptote). The final solution (brown line, Iteration 50000) appears to fit the scatterplot well.

The solution from Iteration 50,000 yielded an R^2 value (coefficient of determination) of 0.661143, which statistically indicates approximately 66.11% of the variance in the dependent variable is explained by the regression model; the model captured a significant portion (but not all) of variability in the data.

5 | Analysis

We have now seen the results of both gradient descent (GD) and simulated annealing (SA). To evaluate the performance of the two optimization algorithms, we will compare their results to an analytical solution calculated using the Levenberg-Marquardt algorithm, a common method for finding very high precision approximations of the exact solution. This algorithm resulted in a theoretical best-fit line for the data which is shown in green in Figure 8.

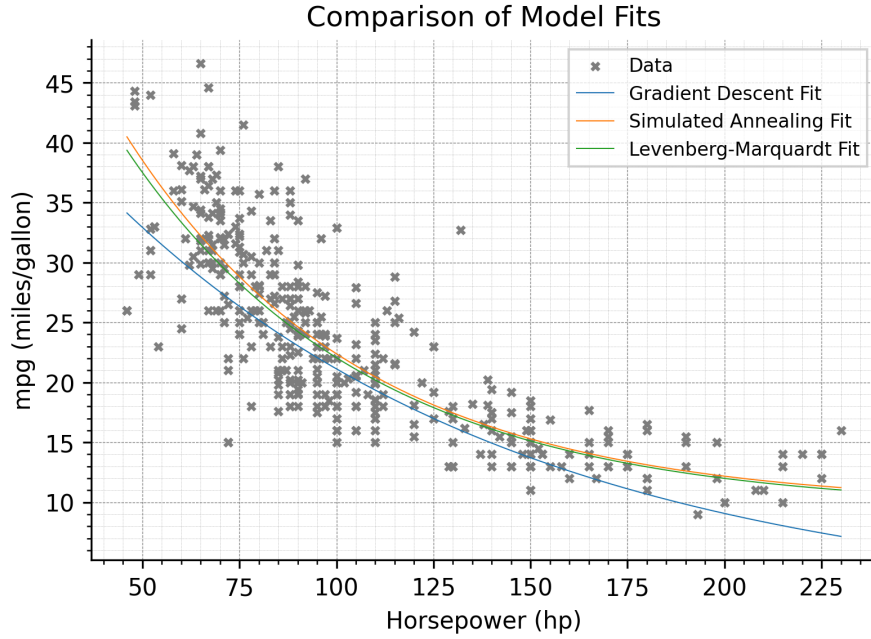


Figure 8: Comparison between two optimizers and the analytical line of best fit

Numerically, the final iteration of each algorithm resulted in the following values:

Algorithm	a	b	h	k	Loss L	R^2 value
Gradient descent	50.010563	-0.009311	1.038900	1.226458	70446.186542	0.603810
Simulated annealing	54.825537	-0.015162	2.979578	10.094089	7579.026930	0.661143
Analytical solution (Levenberg-Marquardt)	41.927	-0.016	24.890	9.453	7514.335123	0.682711

Table 1: Final values for the parameters of the equation, the loss function L , and the coefficient of determination R^2

Comparing the solution found by the two optimization algorithms to the analytical solution suggests that in this problem, simulated annealing performed significantly better than gradient descent.

Simulated annealing was able to find a solution that was closer to the analytical solution (the theoretical best solution), with a difference in the loss value to the analytical solution of 64.691807 compared to gradient descent's orders-of-magnitude higher difference of 62,931.851419. This can be seen visually in Figure 8, where the simulated annealing line of best fit is closer to the analytical solution than the gradient descent line of best fit.

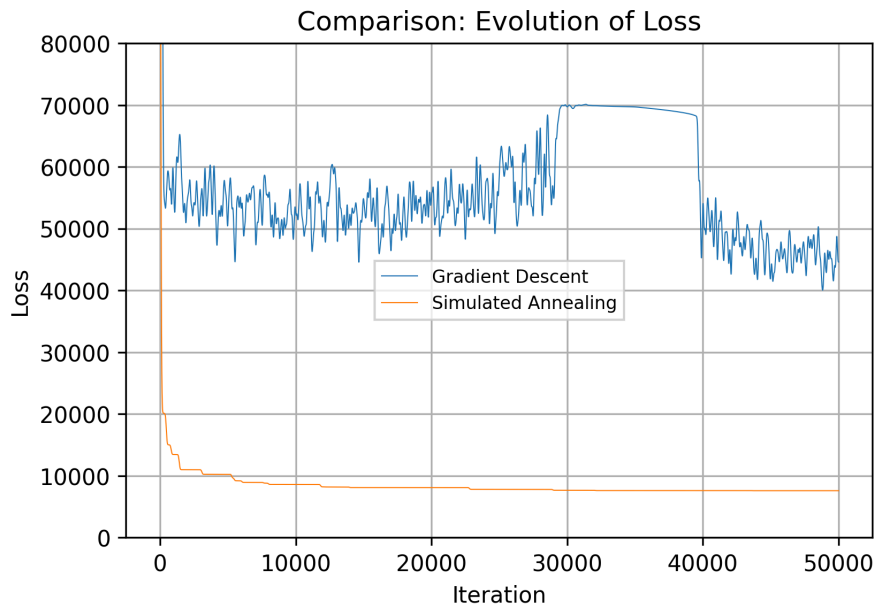


Figure 9: Comparison of loss function over iterations

Looking at the evolution of loss in both algorithms, simulated annealing was able to converge on an optimal solution much faster, reaching this by the 5000th iteration. Gradient descent, on the other hand, took 50,000 iterations to reach a solution that still led to noise in the loss function (it did not converge on a clear minimum).

6 | Conclusion

Not sure what exactly should go here. Summarizing the results would be repetitive?

- could more iterations have been better for gradient descent?
- explain why simulated annealing may have done better: Gradient descent has limitations, especially in cases of non-convex loss functions or when the data has multiple local minima. SA is better at escaping those local minima early on.
- extensions - other optimizers, combinations of methods. Levenberg-Marquardt actually combines gradient descent and Newton's method, which uses 2nd order derivatives.
- what else to put here? need a direct answer to the research question.

Bibliography

Bachman, D. (2007). *Advanced Calculus Demystified*. McGraw-Hill.

Elmetwally, T. (2023). *Car Information Dataset* [Computer software]. <https://www.kaggle.com/datasets/tawfikelmetwally/automobile-dataset>

Freedman, D. A. (2009). *Statistical Models: Theory and Practice*. Cambridge University Press.

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by Simulated Annealing. *Science*, 220(4598), 671–680. <https://doi.org/10.1126/science.220.4598.671>

Sun, S., Cao, Z., Zhu, H., & Zhao, J. (2019, October 23). *A Survey of Optimization Methods from a Machine Learning Perspective*. <http://arxiv.org/abs/1906.06821>

Appendix I: Dataset excerpt

Appendix II: Data processing code and raw outputs